



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس سیستم‌های توزیع شده
گزارش تمرین اول

دانشجویان:

سارا گیتی ۸۱۰۱۰۱۵۰۰

پریا پاسه‌ورز ۸۱۰۱۰۱۳۹۳

فهرست مطالب

۵	فهرست تصاویر
۶	فهرست جداول
۷	۱ پیاده‌سازی ارتباط بین پردازهای با Pipe Named در Go
۷	۱-۱ مقدمه
۸	۲-۱ طراحی کلی سیستم
۸	۳-۱ ساختار Request و Response
۹	۴-۱ عملکرد برنامه Interface
۱۰	۵-۱ عملکرد برنامه Worker
۱۱	۶-۱ مدیریت خطا و پایداری
۱۱	۷-۱ نحوه اجرا
۱۲	۸-۱ نمونه اجرای برنامه
۱۳	۹-۱ تحلیل عملکرد سیستم
۱۵	۱۰-۱ جمع‌بندی
۱۶	۲ همزمانی، زمان‌بندی و بررسی اثرات Switching Context در Go
۱۶	۱-۲ مقدمه
۱۷	۲-۲ طراحی برنامه
۱۹	۱-۲-۲ انواع Workload
۱۹	۲-۲-۲ پارامترهای آزمایش
۲۰	۳-۲ متریک‌های اندازه‌گیری شده
۲۰	۴-۲ نحوه اجرا
۲۰	۱-۴-۲ رسم نمودارها
۲۱	۲-۴-۲ مشاهده Trace
۲۱	۵-۲ نتایج

۲۱	۱-۵-۲ جدول نتایج
۲۶	۲-۵-۲ Throughput نمودار
۲۷	۳-۵-۲ Time Total نمودار
۲۸	۴-۵-۲ Latency Average نمودار
۲۹	۶-۲ تحلیل نتایج
۲۹	۱-۶-۲ با افزایش تعداد goroutine چه اتفاقی برای زمان اجرا می افتد؟
۲۹	۲-۶-۲ آیا افزایش تعداد goroutine همیشه باعث افزایش throughput می شود؟
	۳-۶-۲ تفاوت رفتار برنامه در $GOMAXPROCS = 1$ و $GOMAXPROCS =$
۳۰	$NumCPU$ چیست؟
	۴-۶-۲ در کدام نوع workload اثرات زمان بندی و switching بیشتر دیده
۳۱	می شود؟
	۵-۶-۲ از چه نقطه ای به بعد، افزایش همزمانی مفید نیست یا حتی کارایی را
۳۲	کاهش می دهد؟
۳۲	۷-۲ بررسی Trace
۳۲	۱-۷-۲ Proc by Trace View
۳۳	۲-۷-۲ Thread by Trace View
۳۴	۳-۷-۲ Goroutine Analysis
۳۶	۸-۲ جمع بندی

۳ ساخت سرویس، کانتینری سازی و اجرا در ماشین مجازی ۳۷

۳۷	۱-۳ مقدمه
۳۷	۲-۳ معماری سیستم
۳۷	۳-۳ Endpoint مربوط به Check Health
۳۸	۴-۳ Endpoint مربوط به Compute
۳۸	۵-۳ ساختار Request و Response
۳۹	۶-۳ اعتبارسنجی ورودی

۴۰	 Dockerization ۷-۳
۴۰	 Image Docker Build کردن ۸-۳
۴۱	 Container اجرای ۹-۳
۴۱	 نمونه تست‌های انجام‌شده ۱۰-۳
۴۲	 تست خطاها ۱۱-۳
۴۳	 مزایای استفاده از Docker ۱۲-۳
۴۴	 جمع‌بندی ۱۳-۳

فهرست تصاویر

۱۳	خروجی برنامه worker پس از دو بار اجرا interface	۱
۱۴	خروجی interface و یکبار اتمام برنامه	۲
۱۴	خروجی interface پس از اجرا دوباره	۳
۲۶	CPU Workload	۴
۲۶	IO Workload	۵
۲۶	Mixed Workload	۶
۲۶	Throughput بر حسب تعداد goroutine ها برای workload های مختلف	۷
۲۷	CPU Workload	۸
۲۷	IO Workload	۹
۲۷	Mixed Workload	۱۰
۲۷	زمان کل اجرا بر حسب تعداد goroutine ها برای workload های مختلف	۱۱
۲۸	CPU Workload	۱۲
۲۸	IO Workload	۱۳
۲۸	Mixed Workload	۱۴
۲۸	میانگین latency بر حسب تعداد goroutine ها برای workload های مختلف	۱۵
۳۳	بررسی خروجی trace با توجه به proc	۱۶
۳۴	بررسی خروجی trace با توجه به thread	۱۷
۳۵	Goroutine Analysis	۱۸
۳۵	بررسی زمان اجرای هر یک از goroutines	۱۹
۴۲	خروجی health از مرورگر	۲۰
۴۲	خروجی op از مرورگر	۲۱
۴۲	خروجی سرور در طول اجرا	۲۲
۴۳	خروجی client در ترمینال	۲۳

فهرست جداول

۲۳	 Benchmark Results for CPU Workload	۱
۲۴	 Benchmark Results for IO Workload	۲
۲۵	 Benchmark Results for Mixed Workload	۳

۱ پیاده‌سازی ارتباط بین پردازهای با Pipe Named در Go

۱-۱ مقدمه

در سیستم‌های توزیع‌شده و سیستم‌عامل‌ها، ارتباط بین پردازها (Inter Process Communication - IPC) یکی از مفاهیم اساسی محسوب می‌شود. زمانی که چند برنامه مستقل نیاز به تبادل داده یا هماهنگی داشته باشند، باید از مکانیزم‌هایی برای انتقال اطلاعات میان آن‌ها استفاده شود.

سیستم‌عامل لینوکس روش‌های مختلفی برای IPC فراهم می‌کند که از جمله مهم‌ترین آن‌ها می‌توان به موارد زیر اشاره کرد:

- Pipe
- (FIFO) Pipe Named
- Memory Shared
- Queue Message
- Socket

در این پروژه از Named Pipe استفاده شده است. برخلاف Pipe معمولی که فقط بین پردازهای دارای رابطه والد و فرزند قابل استفاده هستند، Named Pipes به صورت فایل‌هایی در سیستم فایل ایجاد می‌شوند و هر پردازش مستقل می‌تواند به آن متصل شود. هدف این بخش، طراحی یک سیستم ساده Client/Worker برای انجام عملیات محاسباتی با استفاده از Pipe Named در زبان Go است. در این معماری:

- برنامه interface نقش Client را دارد.
- برنامه worker نقش Server را دارد.
- ارتباط بین دو برنامه از طریق FIFO انجام می‌شود.

همچنین در این پروژه تلاش شده است مدیریت خطا، قطع اتصال، reconnect و پایداری سیستم نیز بررسی شود.

۲-۱ طراحی کلی سیستم

سیستم طراحی شده شامل دو پردازش مستقل است:

۱. Process Interface

۲. Process Worker

برای ارتباط بین این دو پردازش، دو Pipe Named مستقل ایجاد شده است:

request_pipe = /tmp/hw1_part1_req_write	1
response_pipe = /tmp/hw1_part1_res_write	2

Pipe اول برای ارسال درخواست از سمت interface به worker استفاده می‌شود و Pipe دوم پاسخ worker را به interface بازمی‌گرداند.

۳-۱ ساختار Request و Response

برای تبادل داده از فرمت JSON استفاده شده است. این کار باعث خوانایی بهتر و توسعه‌پذیری بیشتر سیستم می‌شود.

ساختار درخواست:

type Request struct {	1
Operation string	2
Operand1 int	3
Operand2 int	4
}	5

ساختار پاسخ:

type Response struct {	1
Status string	2
Result int	3
Error string	4
}	5

۴-۱ عملکرد برنامه Interface

برنامه Interface وظایف زیر را بر عهده دارد:

- دریافت ورودی از کاربر
- اعتبارسنجی ورودی
- تبدیل داده‌ها به JSON
- ارسال درخواست به Worker
- دریافت پاسخ
- نمایش نتیجه یا خطا

در ابتدای اجرا، برنامه تلاش می‌کند به های‌پایپ ایجادشده توسط Worker متصل شود. برای جلوگیری از خطا در صورت آماده نبودن Worker مکانیزم Retry پیاده‌سازی شده است. در صورت عدم اتصال موفق پس از چند تلاش، برنامه متوقف می‌شود. فرمت ورودی کاربر به صورت زیر است:

OP A B	1
--------	---

برای مثال:

ADD 5 3

1

برنامه فقط عملیات زیر را قبول می‌کند:

ADD •

SUB •

MUL •

DIV •

۵-۱ عملکرد برنامه Worker

برنامه Worker مسئول انجام عملیات محاسباتی است.

در ابتدای اجرا:

۱. FIFOها ساخته می‌شوند.

۲. Worker منتظر اتصال Interface می‌ماند.

۳. پس از اتصال، درخواست‌ها دریافت می‌شوند.

۴. درخواست پردازش شده و پاسخ ارسال می‌شود.

برای ساخت FIFO از تابع زیر استفاده شده است:

```
syscall.Mkfifo(...)
```

1

پس از دریافت هر درخواست، Worker عملیات موردنظر را اجرا می‌کند.

برای مثال:

ADD • جمع

• SUB تفریق

• MUL ضرب

• DIV تقسیم

در صورت تقسیم بر صفر یا عملیات ناشناخته، خطا برگردانده می‌شود.

۶-۱ مدیریت خطا و پایداری

یکی از اهداف مهم پروژه، پایداری سیستم در شرایط غیرعادی بود. برای این منظور موارد زیر پیاده‌سازی شدند:

• تشخیص Pipe Broken

• تشخیص قطع اتصال Interface

• مدیریت EOF

• مدیریت JSON نامعتبر

• reconnect خودکار Worker

اگر Interface بسته شود، Worker متوقف نشده و دوباره منتظر اتصال Client جدید می‌ماند.

۷-۱ نحوه اجرا

ابتدا Worker اجرا می‌شود:

```
go run worker.go
```

1

خروجی مورد انتظار:

```
[worker] Pipes created. Waiting for interface to connect...1
```

سپس Interface اجرا می‌شود:

```
go run interface.go
```

1

خروجی:

```
[interface] Connecting to worker... 1
[interface] Connected to worker! 2
[interface] Connected! Enter operations as: OP A B 3
[interface] Supported: ADD, SUB, MUL, DIV - type 'exit' to 4
quit.
```

۸-۱ نمونه اجرای برنامه

نمونه اجرای عملیات جمع:

```
> ADD 5 3 1
[interface] Result: 8 2
```

لاگ: Worker:

```
[worker] Received: op=ADD a=5 b=3 1
[worker] Sent: {"status":"ok","result":8} 2
```

نمونه تقسیم:

```
> DIV 10 2 1
[interface] Result: 5 2
```

نمونه تقسیم بر صفر:

```
cts-/1/part1$ go run worker.go
[worker] Pipes created. Waiting for interface to connect...
[worker] Interface connected!
[worker] Ready. Waiting for requests...
2026/05/14 18:47:23 [worker] Received: op=ADD a=5 b=3
2026/05/14 18:47:23 [worker] Sent: {"status":"ok","result":8}
2026/05/14 18:47:34 [worker] Received: op=SUB a=10 b=4
2026/05/14 18:47:34 [worker] Sent: {"status":"ok","result":6}
2026/05/14 18:47:39 [worker] Received: op=MUL a=7 b=6
2026/05/14 18:47:39 [worker] Sent: {"status":"ok","result":42}
2026/05/14 18:47:46 [worker] Received: op=DIV a=20 b=5
2026/05/14 18:47:46 [worker] Sent: {"status":"ok","result":4}
2026/05/14 18:48:12 [worker] Received: op=DIV a=5 b=0
2026/05/14 18:48:12 [worker] Sent: {"status":"err","error":"division_by_zero"}
[worker] Interface disconnected. Waiting for a new connection...
[worker] Restarting pipes...
[worker] Pipes created. Waiting for interface to connect...
[worker] Interface connected!
[worker] Ready. Waiting for requests...
2026/05/14 18:49:58 [worker] Received: op=ADD a=4 b=5
2026/05/14 18:49:58 [worker] Sent: {"status":"ok","result":9}
```

شکل ۱: خروجی برنامه worker پس از دو بار اجرا interface

> DIV 5 0	1
[interface] Error from worker: division_by_zero	2

لاگ Worker:

[worker] Received: op=DIV a=5 b=0	1
[worker] Sent: {"status":"err","error":"division_by_zero"}	2

نمونه عملیات نامعتبر:

> POW 2 3	1
[interface] Error: unknown operation 'POW'. Use ADD SUB MUL DIV	

۹-۱ تحلیل عملکرد سیستم

به دلیل استفاده از Pipe، Named انتقال داده بین پردازها بدون نیاز به شبکه انجام می‌شود و سربار نسبتاً کمی دارد. همچنین استفاده از JSON باعث افزایش خوانایی و سادگی توسعه سیستم شده است، اگرچه نسبت به فرمت‌های باینری کمی سربار بیشتر دارد.

```
/DistributedSystems_Projects-/1/part1$ go run interface.go
[interface] Connecting to worker...
[interface] Connected to worker!
[interface] Connected! Enter operations as: OP A B (e.g. ADD 5 3)
[interface] Supported: ADD, SUB, MUL, DIV - type 'exit' to quit.

> ADD 5 3
[interface] Result: 8

> SUB 10 4
[interface] Result: 6

> MUL 7 6
[interface] Result: 42

> DIV 20 5
[interface] Result: 4

> DIV 5 0
[interface] Error from worker: division_by_zero

> POW 2 3
[interface] Error: unknown operation 'POW'. Use ADD SUB MUL DIV

> ADD x 5
[interface] Error: 'x' is not a valid integer

> ADD 5
[interface] Error: expected format is OP A B (e.g. ADD 5 3)

> exit
[interface] Exiting.
```

شکل ۲: خروجی interface و یکبار اتمام برنامه

```
/DistributedSystems_Projects-/1/part1$ go run interface.go
[interface] Connecting to worker...
[interface] Connected to worker!
[interface] Connected! Enter operations as: OP A B (e.g. ADD 5 3)
[interface] Supported: ADD, SUB, MUL, DIV - type 'exit' to quit.

> ADD 4 5
[interface] Result: 9
```

شکل ۳: خروجی interface پس از اجرا دوباره

مزیت مهم این معماری، جدا بودن کامل Client و Worker است. در نتیجه می‌توان چند Interface مختلف طراحی کرد که همگی به Worker متصل شوند.

۱۰-۱ جمع‌بندی

در این بخش یک سیستم IPC کامل با استفاده از Pipe Named در زبان Go طراحی و پیاده‌سازی شد.

ویژگی‌های مهم سیستم عبارت بودند از:

- استفاده از FIFO

- ارتباط دوطرفه بین پردازها

- استفاده از JSON

- مدیریت خطا

- reconnect خودکار

- ساختار Client/Worker

این پروژه مفاهیم مهم IPC، مدیریت پردازها، و طراحی سیستم‌های مقاوم در برابر خطا را به خوبی نشان می‌دهد.

۲ همزمانی، زمان‌بندی و بررسی اثرات Switching Context در

Go

۱-۲ مقدمه

هدف این بخش، بررسی رفتار اجرای همزمان تعداد زیادی goroutine در زبان Go و تحلیل اثرات زمان‌بندی (Scheduling)، رقابت برای CPU، و سربار ناشی از افزایش concurrency است. در این پروژه برنامه‌ای طراحی شده‌است که با تغییر تعداد goroutine ها، نوع workload و مقدار GOMAXPROCS، متریک‌های مختلف عملکردی را اندازه‌گیری و ثبت می‌کند. همچنین از ابزار runtime/trace برای مشاهده رفتار Scheduler و goroutine ها استفاده شده است.

در زبان go با مفهوم goroutine مواجه هستیم. تفاوت‌های آن با thread را می‌توان در سه جنبه بیان کرد.

۱. **مدیریت و زمان‌بندی:** Thread توسط هسته سیستم‌عامل (OS Kernel) مدیریت می‌شود. جابجایی بین تردها (Context Switch) هزینه بالایی دارد زیرا نیاز به ورود به حالت هسته (Kernel Mode) دارد. در مقابل، Goroutine توسط زمان‌بند خود زبان گو (Go Scheduler) مدیریت می‌شود که در سطح کاربر (User Space) عمل می‌کند. این کار باعث می‌شود هزینه جابجایی بسیار کمتر از تردهای سیستم‌عامل باشد.

۲. **مصرف حافظه:** Thread با یک پشته (Stack) ثابت و نسبتاً بزرگ (معمولاً حدود ۱ تا ۲ مگابایت) شروع می‌شود. این موضوع باعث محدودیت در تعداد تردهای قابل ایجاد در یک سیستم می‌شود. در مقابل، Goroutine با یک پشته بسیار کوچک (حدود ۲ کیلوبایت) شروع می‌شود که بر حسب نیاز به صورت پویا رشد می‌کند یا کوچک می‌شود. به همین دلیل می‌توان میلیون‌ها گوروتین را در یک سیستم معمولی اجرا کرد.

۳. **هزینه جابجایی:** در Thread جابجایی بین دو ترد مستلزم ذخیره و بازیابی تعداد زیادی Registers و عبور از لایه‌های امنیتی هسته است که زمان‌بر است. در مقابل، Goroutine

فقط به ذخیره و بازیابی تعداد کمی ثبات (مانند شمارنده برنامه و اشاره‌گر پشته) نیاز دارد، لذا سرعت جابجایی در آن‌ها بسیار بالاتر است.

Go Scheduler که به مدل M:P:N معروف است، وظیفه مدیریت اجرای گروتین‌ها روی تردهای سیستم‌عامل را بر عهده دارد. برخلاف بسیاری از زبان‌ها که مستقیماً از تردهای سیستم‌عامل استفاده می‌کنند، گو از یک لایه واسط برای بهینه‌سازی استفاده می‌کند تا بتواند میلیون‌ها گروتین را با کمترین هزینه اجرا کند. اجزای اصلی آن عبارتند از:

- G (Goroutine): نشان‌دهنده یک گروتین است که شامل پشته (Stack) و وضعیت اجرای آن می‌باشد.

- M (Machine/OS Thread): نشان‌دهنده یک ترد واقعی سیستم‌عامل است که توسط هسته مدیریت می‌شود.

- P (Processor): یک منبع منطقی است که برای اجرای کد گو مورد نیاز است. تعداد P دقیقاً برابر با مقدار GOMAXPROCS تنظیم می‌شود.

یکی از ویژگی‌های مهم زمان‌بند گو برای حفظ کارایی، الگوریتم Work Stealing است. برای این منظور، هر P یک صف محلی (Local Run Queue) از گروتین‌های منتظر اجرا دارد. اگر یک P تمام کارهای خود را تمام کند و صفش خالی شود، سعی می‌کند نیمی از گروتین‌های صف یک P دیگر را سرقت کرده و اجرا کند. این کار باعث می‌شود تمام هسته‌های CPU همیشه مشغول بمانند و بار کاری متعادل شود.

زمان‌بند گو به‌گونه‌ای طراحی شده که مانع از مسدود شدن کل برنامه در اثر یک System Call شود. وقتی یک گروتین فراخوانی مسدودکننده (مانند خواندن فایل) انجام می‌دهد، ترد M مربوطه مسدود می‌شود. در این حالت، زمان‌بند P را از آن جدا کرده و به یک ترد M جدید یا آماده منتقل می‌کند تا بقیه گروتین‌های صف محلی بتوانند به کار خود ادامه دهند.

۲-۲ طراحی برنامه

ساختار کلی برنامه به صورت یک benchmark framework طراحی شده است.

در هر آزمایش:

۱. تعداد مشخصی task ایجاد می‌شود.
 ۲. تعدادی worker به صورت goroutine اجرا می‌شوند.
 ۳. هر worker یک workload مشخص را اجرا می‌کند.
 ۴. زمان اجرا و latency هر task اندازه‌گیری می‌شود.
 ۵. نتایج در فایل CSV ذخیره می‌شوند.
- برای پیاده‌سازی worker ، از Pool Worker استفاده کردیم. وظیفه این تابع این است که به عنوان یک واحد اجرایی مستقل (در قالب یک گروتین) عمل کرده و وظایفی را از یک کانال ورودی دریافت و پردازش کند.
- در ابتدای تابع، عبارت `wg.Done()` قرار دارد. این دستور تضمین می‌کند که وقتی اجرای تابع به پایان رسید (یعنی کانال jobs بسته شد و حلقه تمام شد)، به `WaitGroup` اطلاع داده شود که این Worker کار خود را انجام داده است. این کار برای هماهنگی و اطمینان از پایان یافتن تمام گروتین‌ها قبل از اتمام برنامه اصلی ضروری است. حلقه تا زمانی که کانال jobs باز باشد، به گوش دادن ادامه می‌دهد. هر زمان که عددی در کانال قرار بگیرد، حلقه یک تکرار انجام می‌دهد. این مدل اجازه می‌دهد که چندین worker به صورت همزمان از یک کانال مشترک داده بردارند، بدون اینکه یک داده توسط دو worker پردازش شود. در نهایت نیز مقدار latency محاسبه شده و به کانال ارسال می‌شود.
- در قدم بعدی می‌خواهیم Benchmark را اجرا کنیم. اول تعداد پردازنده‌ی منطقی (P) را مشخص می‌کنیم، بعد Worker Pool را راه‌اندازی می‌کنیم، یعنی به سیستم اعلام می‌کنیم که یک واحد اجرایی جدید شروع به کار کرده است و هر worker در یک گروتین مجزا شروع به گوش دادن به کانال jobs می‌کند. پس از این مرحله، برنامه اصلی تمام تسک‌ها را داخل کانال می‌ریزد و سپس کانال را می‌بندد. بستن کانال یک سیگنال به تمام کارگراهاست که یعنی دیگر کاری باقی نمانده و آن‌ها می‌توانند پس از اتمام کارهای فعلی، از حلقه خارج شوند. برنامه اصلی اینجا متوقف می‌شود تا تمام کارگراها کارشان تمام شود و `wg.Done()` را صدا بزنند.

۲-۲-۱ انواع Workload

در این آزمایش‌ها سه نوع workload مختلف پیاده‌سازی کردیم:

۱. CPU-bound

در این بخش می‌خواهیم محاسبات سنگین برای بررسی فشار روی CPU و Scheduler داشته باشیم. برای این منظور در یک حلقه تمام اعداد اول کوچکتر از ۲۰۰۰۰ را محاسبه می‌کنیم.

۲. IO-bound

عملیات IO-Bound را با استفاده از time.Sleep شبیه‌سازی کردیم.

۳. Mixed

در اینجا ترکیبی از workload محاسباتی و blocking داریم. اول cpuWork انجام می‌شود سپس sleep خواهیم داشت.

۲-۲-۲ پارامترهای آزمایش

در آزمایش‌ها سه پارامتر اصلی تغییر داده شدند:

• تعداد goroutine ها:

۱، ۲، ۴، ۸، ۱۶، ۳۲، ۶۴، ۱۲۸

• مقدار GOMAXPROCS:

۱، ۲، runtime.NumCPU()

• نوع worload :

cpu io mixed

۳-۲ متریک‌های اندازه‌گیری‌شده

متریک‌های زیر برای هر اجرای benchmark ثبت شدند:

- زمان کل اجرا
- throughput
- میانگین latency
- حداقل latency
- حداکثر latency
- انحراف معیار latency

۴-۲ نحوه اجرا

ابتدا وارد پوشه پروژه می‌شویم:

```
cd part2
```

1

اجرای benchmark:

```
go run main.go
```

1

پس از اجرا، فایل‌های results/benchmark.csv و trace.out تولید می‌شوند

۱-۴-۲ رسم نمودارها

برای رسم نمودارها از Python استفاده شد.

ابتدا:

```
pip install pandas matplotlib
```

1

سپیس:

```
python3 plot_results.py
```

1

۲-۴-۲ مشاهده Trace

برای مشاهده رفتار Scheduler :

```
go tool trace trace.out
```

1

این ابزار امکان مشاهده موارد وضعیت goroutine ها ، زمان‌بندی پردازنده‌ها ، blocking و فعالیت Collector Garbage را فراهم می‌کند.

۲-۵ نتایج

۲-۵-۱ جدول نتایج

در این قسمت نتایج اجرای برنامه را مشاهده می‌کنیم.

نتایج به‌دست‌آمده نشان‌دهنده نحوه مدیریت انواع بار کاری توسط زمان‌اجرای go در سطوح مختلف همزمانی و موازی‌سازی است. برای کارهای وابسته به پردازنده، زمانی که مقدار GO-MAXPROCS مطابق با رشته‌های سخت‌افزاری موجود افزایش می‌یابد، کارایی به‌طور مؤثری مقیاس‌پذیر می‌شود؛ برای نمونه، زمانی که تعداد پردازنده‌ها از ۱ به ۴ (در حالت ۴ گروتین) افزایش می‌یابد، توان عملیاتی (Throughput) تقریباً سه برابر شده و از ۲۳۵/۴۰ به ۷۸۶/۴۷ می‌رسد. با این حال، افزایش تعداد گروتین‌ها فراتر از تعداد پردازنده‌های در دسترس، منجر به افزایش چشمگیر بار اضافی زمان‌بندی Scheduling Overhead می‌شود؛ به‌طوری که میانگین تاخیر با رسیدن تعداد گروتین‌ها به ۱۲۸، از ۴۲۴۵/۴۷ μs به بیش از ۱۰۰,۰۰۰ μs جهش می‌کند. این موضوع نمونه‌ای از Resource Contention است که در آن گروتین‌ها زمان بیشتری را به جای اجرا، در صف انتظار سپری می‌کنند.

در مقابل، بار کاری وابسته به ورودی/خروجی افزایش (I/O-bound) شدیدی در توان عملیاتی را با بالا رفتن تعداد گروتین‌ها نشان می‌دهد، صرف‌نظر از اینکه مقدار GOMAXPROCS چقدر

باشد. از آنجا که وظایف I/O غیرمسدودکننده هستند یا شامل انتظار می‌شوند، زمان‌بند گو می‌تواند در طول زمان‌های انتظار، روی گوروتین‌های دیگر سوئیچ کند (Context Switch)؛ این امر باعث می‌شود توان عملیاتی با ۱۲۸ گوروتین روی تنها یک پردازنده، از ۲۹۵/۱۲ به ۳۱,۴۷۹/۸۸ صعود کند. بار کاری Mixed نیز در نقطه‌ای بین این دو حالت قرار می‌گیرد و از هر دو مزیت موازی‌سازی و همزمانی بالا بهره می‌برد. به‌طور کلی، داده‌ها تایید می‌کنند که در حالی که برنامه‌های مبتنی بر I/O برای پنهان کردن تاخیر به تعداد گوروتین بالایی نیاز دارند، وظایف محاسباتی سنگین نیازمند هم‌ترازی دقیق با هسته‌های فیزیکی هستند تا از افت عملکرد ناشی از تعویض متن بیش از حد و انحراف معیار بالای تاخیر جلوگیری شود.

جدول :! CPU Workload for Benchmark Results

Us Latency StdDev	Us Latency Max	Us Latency Min	Us Latency Avg	Throughput	Ms Time Total	Gomaxprocs	Goroutines
٨٤٤/٧٢	٢٣٨٠٠/٠٠	٤٠٠٩/٠٠	٤٢٣٨/٨٧	٢٣٥/٧٩	٤٢٤١/٠٠	١	١
٨٤٤٩/٥٥	٤٨٠٤٥/٠٠	٤٠١١/٠٠	٨٤٤١/٨١	٢٣٤/٢٤	٤٢٣٢/٠٠	١	٢
٢٧١٤٨/٢١	٩٥٩٩٤/٠٠	٣٩٩٤/٠٠	١٨٧٤٢/٢٠	٢١٢/٢٤	٤٧١١/٠٠	١	٤
٥٩٧٢٠/٤٥	١٨٠٠٩٤/٠٠	٤٠٠٩/٠٠	٣٢٧٧١/٥٠	٢٣٩/٣٠	٤١٧٨/٠٠	١	٨
١٣١٩٨٤/٧١	٣٥٧٤٤٧/٠٠	٤٠٠٧/٠٠	٧١٢٧٢/٤٤	٢١٥/٨٠	٤٤٣٤/٠٠	١	١٤
٢٧٤٠٧٢/٩٤	٧١٤٢٨٤/٠٠	٤٠٢٧/٠٠	١٥٢٠٠٥/٩٧	١٩٥/٢٧	٥١٢١/٠٠	١	٣٢
٥٠٧٢٩٤/٨٢	٢٥٥٥٢٨٤/٠٠	٤٠٢١/٠٠	٢٢٣٨٣٥/٠٤	٢١٨/٤٠	٤٥٧٨/٠٠	١	٤٤
٨٤١٢٧١/٥١	٣٢٥٢٤٣٥/٠٠	٤٠١٢/٠٠	٣٨٧٤٩٠/٢٤	٢١٩/٠٢	٤٥٤٥/٠٠	١	١٢٨
٤٠٣/٤٤	٨٤٩٥/٠٠	٤٠١٩/٠٠	٤٤٥٤/٢٢	٢٢٤/٢٢	٤٤٥٩/٠٠	٢	١
١٠٨٥/٩٠	١٢٨٩٤/٠٠	٤٠٢٢/٠٠	٥٢٤١/٣٤	٣٨١/٢٣	٢٤٢٣/٠٠	٢	٢
٩٨٥٤/٩٤	٤٩٤٤١/٠٠	٤٠٠٤/٠٠	٩٠١٠/٧١	٤٤١/٧٤	٢٢٤٣/٠٠	٢	٤
٢٨٥٢٤/٩٧	١٣١٩٨٤/٠٠	٤٠١١/٠٠	١٧٤٩١/١٢	٤٤٤/٣٣	٢٢٥٠/٠٠	٢	٨
٤٥٢٥٠/٠٤	٣٧٤٩٨١/٠٠	٤٠٠٧/٠٠	٣٤١٩٥/٨٥	٤٢٤/٤٧	٢٣٤٣/٠٠	٢	١٤
١٢٧٥٠١/٤٤	٤١٣٨٠٤/٠٠	٣٩٨٩/٠٠	٤٢٥٥٨/٧٣	٤٤٩/٢٠	٢١٣١/٠٠	٢	٣٢
٢٤٢٧١٤/١٤	١٠٣٠٢٨٤/٠٠	٤٠٠٧/٠٠	١١٠٨٩٨/٣٢	٤٧٤/٥٤	٢١٠٧/٠٠	٢	٤٤
٤١٤١١٨/٥٤	١٩٣٥٩٤٠/٠٠	٤٠٠٤/٠٠	١٩٤٠٧٨/٠١	٤٢١/١٠	٢٣٧٤/٠٠	٢	١٢٨
٥٢٤/٨٠	١٣٩٠٠/٠٠	٤٠٠٠/٠٠	٤٢٢٤٥/٤٧	٢٣٥/٤٠	٤٢٤٨/٠٠	٤	١
٧٣٢/٨٢	٩٢٧٣/٠٠	٣٩٩٠/٠٠	٤٥٥٥/٠٤	٤٣٨/٥٤	٢٢٨٠/٠٠	٤	٢
١٤٧٤/٩٤	١٤٧٥٨/٠٠	٤٠١٠/٠٠	٥٠٧٤/٠٠	٧٨٤/٤٧	١٢٧١/٠٠	٤	٤
١١٤٨٣/٤٩	٧٠٩١١٤/٠٠	٤٠٠٧/٠٠	٩٣٢٣/٥٧	٨٥١/٠٣	١١٧٥/٠٠	٤	٨
٢٨٤٧٨/٥٥	١٥٧٠٨٠/٠٠	٤٠٠٧/٠٠	١٩٢٣٤/٨١	٨٠٤/٤٥	١٢٤٣/٠٠	٤	١٤
٤١٠٤٥/٠٤	٢٢٤٨٢٤/٠٠	٤٠١٠/٠٠	٣٤٨٢٧/٩٥	٨٢٩/٨٨	١٢٠٤/٠٠	٤	٣٢
١١٧٢٨٢/٧٢	٥٥٤٢٣٣١/٠٠	٤٠٠٩/٠٠	٤٠٠٥٩/٩٣	٨٣٣/٧٠	١١٩٩/٠٠	٤	٤٤
٢٢٢٤٠٧٨/٣٤	١١١٥٠١/٠٠	٣٩٨٩/٠٠	١٠٠٩٩٩/٤٧	٨٤٩/٩٩	١١٧٤/٠٠	٤	١٢٨

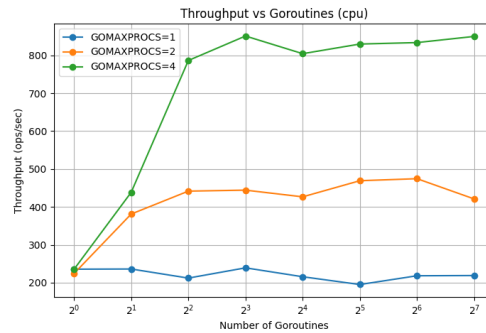
جدول ٢: Benchmark Results for IO Workload

Us Latency StdDev	Us Latency Max	Us Latency Min	Us Latency Avg	Throughput	Ms Time Total	Gomaxprocs	Goroutines
٨٢٧,٧٢	٤٩٧٥,٠٠	٢٠٠٣,٠٠	٣٣٨٥,٤٤	٢٩٥,١٢	٣٣٨٨,٠٠	١	١
٨١٤,٠٤	٤٣٢٥,٠٠	٢٠٠٣,٠٠	٣٣٥٢,٠٨	٥٩٤,٢٧	١٤٧٧,٠٠	١	٢
٨٤٨,٨١	٤١٠٩,٠٠	٢٠٠٢,٠٠	٣٣٥٣,٧٤	١١٩٢,٣٠	٨٣٨,٠٠	١	٤
٨٠٠,٩٥	٤١٤٥,٠٠	٢٠٠١,٠٠	٣٣٩٤,٢٤	٢٣٣٥,٢٠	٤٢٨,٠٠	١	٨
٨٣٩,٤٥	٤٠٤٣,٠٠	٢٠٠١,٠٠	٣٣٩٣,٨٧	٤٤٤٧,٨٠	٢١٤,٠٠	١	١٤
٧٣٤,٠٠	٤٤٩٨,٠٠	٢٠٠١,٠٠	٣٥٤٩,٢٤	٨٧٢٥,٠١	١١٤,٠٠	١	٣٢
٥١٧,٤٣	٤٠٧٤,٠٠	٢٠٠١,٠٠	٣٧٨٩,٧٤	١٤٤٣,٤٩	٤٠,٠٠	١	٤٤
٢٢٨,٤٥	٤١٩٤,٠٠	٣٢٢٥,٠٠	٣٩٢١,٥٢	٣١٤٧٩,٨٨	٣١,٠٠	١	١٢٨
٤٧٥,٢٥	٤٩٢٨,٠٠	٢٠٠٥,٠٠	٣٥٨٤,٩٩	٢٧٨,٥٤	٣٥٨٩,٠٠	٢	١
٤٧٣,٢٣	٥٠٤٢,٠٠	٢٠٠٣,٠٠	٣٥٩٣,٨٠	٥٥٤,٠١	١٧٩٨,٠٠	٢	٢
٤٤٢,١٨	٤٩٢٤,٠٠	٢٠٠٤,٠٠	٣٥٩٧,٤٤	١١١٠,٣٠	٩٠٠,٠٠	٢	٤
٧٠٢,٩٣	٤٣٩٨,٠٠	٢٠٠٣,٠٠	٣٥٤٢,١٥	٢٢٤٢,٢٣	٤٤٥,٠٠	٢	٨
٨٢١,٥٠	٤٠٩٨,٠٠	٢٠٠٨,٠٠	٣٣١٨,٣٠	٤٧٧١,٤٠	٢٠٩,٠٠	٢	١٤
٥٣٨,٢٧	٤١٨٣,٠٠	٢٠٣٩,٠٠	٣٧٥٧,٠٩	٨٢٩١,٤٩	١٢٠,٠٠	٢	٣٢
٤١٠,٤٤	٤٢١٤,٠٠	٢٠٤٨,٠٠	٣٤٧٤,٤٠	١٤٨٨٣,٧٢	٥٩,٠٠	٢	٤٤
٤٤٨,٤٣	٤١٤٠,٠٠	٢٠٠١,٠٠	٣٤٣١,٥٥	٣٤٧٠٠,٤٨	٢٨,٠٠	٢	١٢٨
٤٠١,١٩	٤٤٣١,٠٠	٢٠٢٧,٠٠	٣٤٤٢,٣٣	٢٧٢,٤٧	٣٤٤٧,٠٠	٤	١
٧١٤,٥١	٤٧٤٤,٠٠	٢٠١٤,٠٠	٣٤٨٤,١٥	٥٧٢,٩٩	١٧٤٥,٠٠	٤	٢
٧٣٤,٧٤	٤٣٢٧,٠٠	٢٠٠٣,٠٠	٣٥١٨,٧٧	١١٣٤,٠٩	٨٨٠,٠٠	٤	٤
٧٥٢,٩٥	٤٣٤٣,٠٠	٢٠٢٥,٠٠	٣٤٤٥,٨٩	٢٢٨٩,١٣	٤٣٤,٠٠	٤	٨
٧٠٩,٢٤	٤٥٠١,٠٠	٢٠٧٤,٠٠	٣٤٧٤,٤٧	٤٥٥٨,٩٤	٢١٩,٠٠	٤	١٤
٧٥٧,٨١	٤٧٣٣,٠٠	٢٠٠١,٠٠	٣٥٤٤,٨٨	٨٧٣٢,٢٩	١١٤,٠٠	٤	٣٢
٣٠٨,٢١	٤١٩٧,٠٠	٢٠٢٥,٠٠	٣٧٤٥,٨٣	١٤٤٠,٤٤	٤٠,٠٠	٤	٤٤
٩٤٢,٣٢	٥٨٤٩,٠٠	٢٠٠٧,٠٠	٣٢٨٠,٥٩	٣٥٥١٠,٠٤	٢٨,٠٠	٤	١٢٨

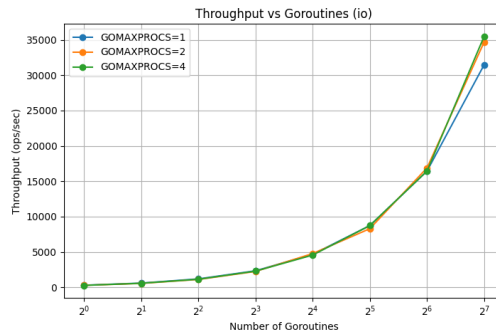
جدول ٣: Benchmark Results for Mixed Workload

Us Latency StdDev	Us Latency Max	Us Latency Min	Us Latency Avg	Throughput	Ms Time Total	Gomaxprocs	Goroutines
٣٥١,٠٨	١٢٥٥٠,٠٠	٥١٨٤,٠٠	٤٠٠٤,٨٨	١٤٤,٤٨	٤٠٠٤,٠٠	١	١
١٧٢٤,٨٧	١٨٤٩٤,٠٠	٥١٠٤,٠٠	٨٥٧٧,٢٧	٢٣٢,٩٧	٤٢٩٢,٠٠	١	٢
١٧٤٨٩,٥٩	٩٥٢١٧,٠٠	٥٠٧٨,٠٠	١٤٣٥٥,٥١	٢٤٣,٢٤	٤١١١,٠٠	١	٤
٥٠٠٧,١٠	٢٧٧٤٣٧,٠٠	٥١٠٣,٠٠	٣٢٤٤٥,٤٢	٢٤٣,٣٤	٤١٠٩,٠٠	١	٨
١١٤١٠٨,٥٨	٥٤٣٥٥٥٢,٠٠	٥٠٣٤,٠٠	٤٥٢٣٨٤,٨٣	٢٣٥,٧٧	٤٢٢٤١,٠٠	١	١٤
٢٢٥٥٢٣١,٠٩	١١٤٩١٤٣,٠٠	٥٠٥٥٣,٠٠	١٢٢٥٣٥,٤٩	٢٣٩,٤٣	٤١٧٣,٠٠	١	٣٢
٤٤٩٤٤٣,٩٣	٢١٢٠٠٢٢,٠٠	٥٠٤٨,٠٠	٢٢١٩٨٨,٥٧	٢٤٠,٥٠	٤١٥٨,٠٠	١	٤٤
٧٩٨١٥٠,٨٧	٣٧٩٤٤٣٢,٠٠	٥١٤٤,٠٠	٢٥٣٠٣٢,٤٧	٢٣٩,١٤	٤١٨١,٠٠	١	١٢٨
٤٧٩,١١	١٤٤٢١,٠٠	٥١٣٤,٠٠	٤١٢٥,٧٨	١٤٣,١٧	٤١٢٨,٠٠	٢	١
٤٧٠,١٤	١١٥٧٥,٠٠	٥٠٨٤,٠٠	٤١٣٧,٩٩	٣٢٥,٥٧	٣٠٧١,٠٠	٢	٢
١٤٤٢,٤٥	١٤٢٩٩,٠٠	٥٠٧٠,٠٠	٨٣٨٨,٢٤	٤٧٥,٩٧	٢١٠٠,٠٠	٢	٤
١٨٤٣٩,٣٣	١٠٢٧٠٥,٠٠	٥٠٧٤,٠٠	١٤٢٢٠,٧٤	٤٨٥,١١	٢٠٤١,٠٠	٢	٨
٤٩٥٧٩,٤٣	٢٥٨٧٤٣,٠٠	٥٠٧٩,٠٠	٣١٨٢٢,١٥	٤٨٥,٤٧	٢٠٥٩,٠٠	٢	١٤
١١٢٤٤٤,٥٢	٤٨٧٥٣٠,٠٠	٥٠٥٥,٠٠	٤١٥٨٧,٤٣	٤٨٠,٨٩	٢٠٧٩,٠٠	٢	٣٢
٢٤٨٧٢٩,٠٢	١٣٢٢١٠,٠٠	٥٢١٤,٠٠	١٣٤١٣٠,٣٤	٤٠٧,٠٠	٢٤٥٧,٠٠	٢	٤٤
٤٢٥٤٤٩٠,٧١	٢١١٤٩٩٢,٠٠	٥٠٧٨,٠٠	٢١٢٩٨٥,٠٤	٤٢٥,٨٣	٢٣٤٨,٠٠	٢	١٢٨
٥٤٩,٣٣	١٣٠٩٢,٠٠	٥١٤٢,٠٠	٤١٧٩,٧٢	١٤١,٧٣	٤١٨٣,٠٠	٤	١
٥٩٢,٠٤	٩١٤٠,٠٠	٥١٠٣,٠٠	٤٢٤٧,٣٥	٣١٩,٨٤	٣١٢٤,٠٠	٤	٢
١١٤١,٤١	١٣٩٨١,٠٠	٥٠٤٨,٠٠	٤٤٩٨,٩٣	٥٩٤,٥٠	١٤٧٤,٠٠	٤	٤
٣٣٢٤٥,٢٤	٢٣٢٢٤,٠٠	٥١٧٧,٠٠	١٠٧٧٥,٧٤	٧٣٧,٧٢	١٣٥٥,٠٠	٤	٨
٢١٠٠,١٤	١١٠١٩٩,٠٠	٥٠٤٨,٠٠	٢١٤١٨,١٤	٧٣٣,٩٨	١٣٤٢,٠٠	٤	١٤
٥٢٣٠٩,٤٧	٢٥١٩٣٩,٠٠	٥٠٣١,٠٠	٣٤٢٥٢,٤٤	٨١٨,٠٩	١٢٢٢,٠٠	٤	٣٢
٩٩٤٠٩,٨٢	٧٧٢٤٥٠,٠٠	٥٠٤٧,٠٠	٥٤٤٥٠,٤٤	٨٤٣,٤٩	١١٨٥,٠٠	٤	٤٤
١٧١٧٤٢,٤٠	٨٤٧٤٤٤,٠٠	٥٠٩٣,٠٠	٨٧٢٤٨,١٣	٨٤٧,٣١	١١٥٢,٠٠	٤	١٢٨

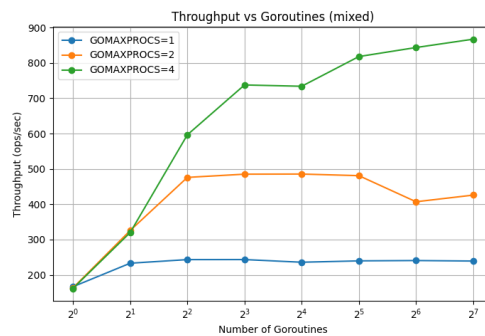
۲-۵-۲ نمودار Throughput



شکل ۴: CPU Workload



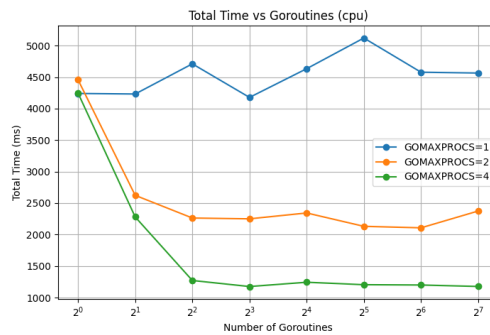
شکل ۵: IO Workload



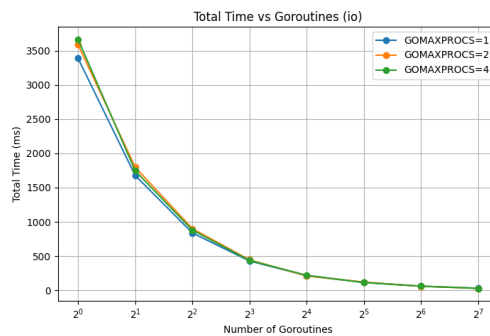
شکل ۶: Mixed Workload

شکل ۷: Throughput بر حسب تعداد goroutine برای workloadهای مختلف

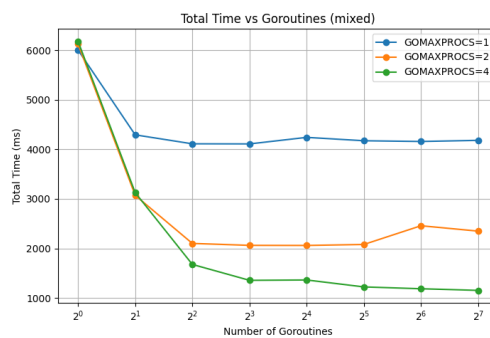
۳-۵-۲ نمودار Time Total



شکل ۸: CPU Workload



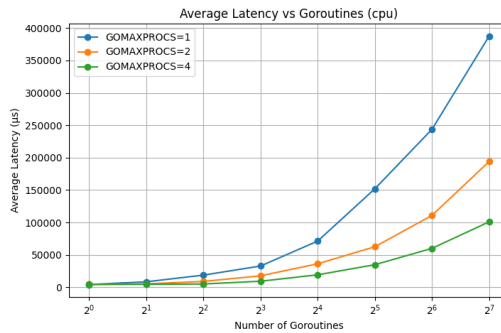
شکل ۹: IO Workload



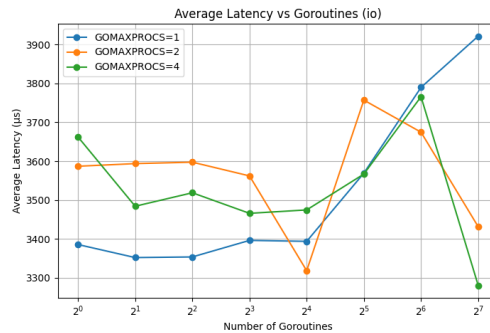
شکل ۱۰: Mixed Workload

شکل ۱۱: زمان کل اجرا بر حسب تعداد goroutine برای workloadهای مختلف

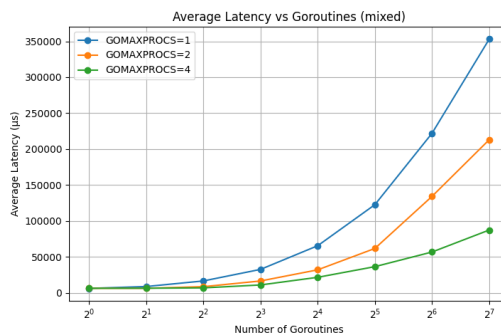
۲-۵-۴ نمودار Latency Average



شکل ۱۲: CPU Workload



شکل ۱۳: IO Workload



شکل ۱۴: Mixed Workload

شکل ۱۵: میانگین latency بر حسب تعداد goroutine ها برای workload های مختلف

۶-۲ تحلیل نتایج

۱-۶-۲ با افزایش تعداد goroutine چه اتفاقی برای زمان اجرا می‌افتد؟

پاسخ به این سوال بستگی مستقیمی به نوع بار کاری (Workload) و میزان موازی‌سازی (GOMAXPROCS) دارد:

۱. در بارهای کاری CPU-bound :

اگر تعداد گروتین‌ها از تعداد هسته‌های فیزیکی (GOMAXPROCS) فراتر رود، زمان اجرای کل (Total Time) لزوماً بهبود نمی‌یابد و حتی ممکن است به دلیل Scheduling Overhead و رقابت بر سر منابع، افزایش یابد. در این حالت، میانگین تاخیر Avg Latency به شدت افزایش می‌یابد؛ زیرا گروتین‌ها باید در صف انتظار بمانند تا نوبت اجرای آن‌ها روی هسته‌های محدود برسد.

۲. در بارهای کاری I/O-bound :

با افزایش تعداد گروتین‌ها، زمان اجرای کل کاهش چشمگیری می‌یابد. به دلیل ماهیت غیرمسدودکننده I/O در گو، زمان‌بند می‌تواند در حالی که یک گروتین منتظر پاسخ شبکه یا دیسک است، گروتین دیگری را اجرا کند. این امر باعث استفاده بهینه از زمان و افزایش خیره‌کننده توان عملیاتی (Throughput) می‌شود.

۳. در بارهای کاری Mixed :

زمان اجرا تا یک نقطه بهینه کاهش می‌یابد و پس از آن، افزایش بیش از حد گروتین‌ها منجر به افزایش انحراف معیار تاخیر (StdDev Latency) و ناپایداری در زمان پاسخ‌گویی می‌شود.

۲-۶-۲ آیا افزایش تعداد goroutine همیشه باعث افزایش throughput می‌شود؟

خیر، افزایش تعداد گروتین‌ها همیشه باعث افزایش توان عملیاتی (Throughput) نمی‌شود. این موضوع به نوع بار کاری و ظرفیت منابع سیستم وابسته است:

۱. در بارهای کاری CPU-bound :

وقتی تعداد گروتین‌ها از تعداد هسته‌های فیزیکی (GOMAXPROCS) فراتر می‌رود، توان عملیاتی به یک سطح اشباع می‌رسد و سپس ممکن است کاهش یابد. دلیل این اتفاق، سربار ناشی از تعویض متن (Context Switching) و رقابت برای کش پردازنده است. زمان‌بند باید مدام بین گروتین‌ها جابجا شود بدون اینکه کار مفید بیشتری انجام شود.

۲. در بارهای کاری I/O-bound :

در این سناریو، افزایش گروتین‌ها معمولاً باعث افزایش چشمگیر توان عملیاتی می‌شود، زیرا زمان‌های انتظار (Latency) با اجرای کارهای دیگر توسط گروتین‌های موازی پنهان می‌شود. اما حتی در اینجا هم یک حد نهایی وجود دارد؛ اگر تعداد گروتین‌ها بیش از حد زیاد شود، مصرف حافظه (Stack) و سربار مدیریت لیست‌های انتظار در زمان‌بند گو، توان عملیاتی را تحت تاثیر قرار می‌دهد.

۳. در بارهای کاری Mixed :

طبق قانون امدال (Amdahl's Law)، بخش‌های ترتیبی کد (بخش‌هایی که نمی‌توانند موازی شوند، مثل دسترسی به یک منبع مشترک) اجازه نمی‌دهند که با افزایش بی‌رویه گروتین‌ها، توان عملیاتی به طور خطی رشد کند.

۳-۶-۲ تفاوت رفتار برنامه در $GOMAXPROCS = 1$ و $GOMAXPROCS = NumCPU$ چیست؟

تفاوت اصلی این دو تنظیمات در جابجایی از مدل Concurrency به مدل Parallelism است.

در حالت $GOMAXPROCS = 1$ ، بدون توجه به تعداد هسته‌های پردازنده، تنها یک هسته به اجرای گروتین‌ها اختصاص می‌یابد و گروتین‌ها به صورت Interleaved اجرا می‌شوند؛ یعنی پردازنده مدام بین آن‌ها سوئیچ می‌کند (Time-slicing). برای کارهای CPU-bound، افزایش تعداد گروتین‌ها در این حالت بهبودی در زمان اجرا ایجاد نمی‌کند و صرفاً باعث بالا رفتن میانگین

تاخیر (*AvgLatency*) می‌شود، زیرا همواره فقط یک کار در لحظه در حال انجام است. در حالت $GOMAXPROCS = NumCPU$ ، زمان اجرای گو اجازه می‌یابد از تمام توان سخت‌افزاری پردازنده استفاده کند و در این حالت، چندین گروتین می‌توانند دقیقاً در یک لحظه واحد روی هسته‌های مختلف اجرا شوند. در این حالت، اگرچه سرعت کل افزایش می‌یابد، اما مدیریت دسترسی به منابع مشترک و هماهنگی بین هسته‌ها (Synchronization) اهمیت پیدا می‌کند.

۲-۶-۴ در کدام نوع workload اثرات زمان‌بندی و switching بیشتر دیده می‌شود؟

اثرات Scheduling و Context Switching به طور متفاوتی در انواع بار کاری ظاهر می‌شوند، اما شدیدترین اثرات منفی آن در بار کاری محاسباتی (CPU-bound) دیده می‌شود.

۱. در بارهای کاری CPU-bound :

در این نوع بار کاری، گروتین‌ها به طور مداوم از چرخه پردازنده استفاده می‌کنند. وقتی تعداد گروتین‌ها زیاد می‌شود (بسیار بیشتر از تعداد هسته‌ها)، زمان‌بند ناچار است برای رعایت Fairness، اجرای یک گروتین محاسباتی را متوقف کرده و گروتین دیگری را جایگزین کند. این کار باعث خالی شدن کش پردازنده (Cache Miss) و صرف زمان قابل توجهی برای ذخیره و بازیابی وضعیت registers می‌شود. افزایش شدید انحراف معیار تاخیر (StdDev Latency) در حالت CPU (از ۸۶۶ به ۲۷۱، ۸۶۱ در ۱۲۸ گروتین) نشان‌دهنده همین بی‌نظمی و سربار ناشی از سوئیچینگ مداوم است.

۲. در بارهای کاری I/O-bound :

در کارهای ورودی/خروجی، سوئیچینگ یک ضرورت است نه یک بار اضافی. وقتی یک گروتین منتظر شبکه یا دیسک می‌ماند، داوطلبانه پردازنده را رها می‌کند. اگرچه تعداد سوئیچ‌ها در اینجا بسیار زیاد است، اما چون این سوئیچینگ در زمان‌های بیکاری پردازنده رخ می‌دهد، باعث بهبود کارایی و بالا رفتن توان عملیاتی می‌شود. به همین دلیل است که در داده‌های شما، توان عملیاتی IO با افزایش همزمانی به شدت رشد کرده است.

۲-۶-۵ از چه نقطه‌ای به بعد، افزایش همزمانی مفید نیست یا حتی کارایی را کاهش می‌دهد؟

نقطه اشباع یا بازده نزولی (Saturation Point) در یک سیستم، زمانی رخ می‌دهد که منابع به حداکثر ظرفیت خود برسند.

۱. در بارهای کاری CPU-bound :

افزایش همزمانی زمانی غیرمفید می‌شود که تعداد گوروتین‌های فعال از تعداد هسته‌های فیزیکی در دسترس (GOMAXPROCS) بیشتر شود. طبق داده‌ها، با افزایش گوروتین‌ها از ۶۴ به ۱۲۸ در حالت $GOMAXPROCS = ۱$ ، توان عملیاتی تقریباً ثابت مانده ($۲۱۸/۴۰ \rightarrow$ ۲۱۹/۰۲) اما میانگین تاخیر بیش از ۵۰٪ رشد کرده است. در این نقطه، سیستم وارد فاز Thrashing می‌شود که در آن بخش بزرگی از توان پردازنده صرف تعویض متن و مدیریت صف‌های زمان‌بند به جای انجام محاسبات واقعی می‌شود.

۲. در بارهای کاری I/O-bound :

نقطه اشباع بسیار دیرتر رخ می‌دهد، زیرا گلوگاه در اینجا قدرت پردازنده نیست، بلکه پهنای باند شبکه یا سرعت پاسخ‌دهی دیسک است. با این حال، افزایش همزمانی از نقطه‌ای به بعد مفید نیست که هزینه مدیریت حافظه برای گوروتین‌های جدید (هر کدام حداقل ۲KB حافظه پشته نیاز دارند) و سربار مدیریت لیست‌های انتظار (Waiting Queues) از سود ناشی از موازی‌سازی انتظار بیشتر شود.

۲-۷ بررسی Trace

۲-۷-۱ Proc by Trace View

در این قسمت مشاهده می‌کنیم که تعداد گوروتین‌ها به صورت پله‌ای افزایش می‌یابد. این معادل همان لحظاتی است که کد در حال اجرای حلقه for برای ساخت گوروتین‌های worker هستیم.

در قسمتی دیگر میزان مصرف حافظه (heap) مشاهده می‌شود. قله‌های تیز و سپس



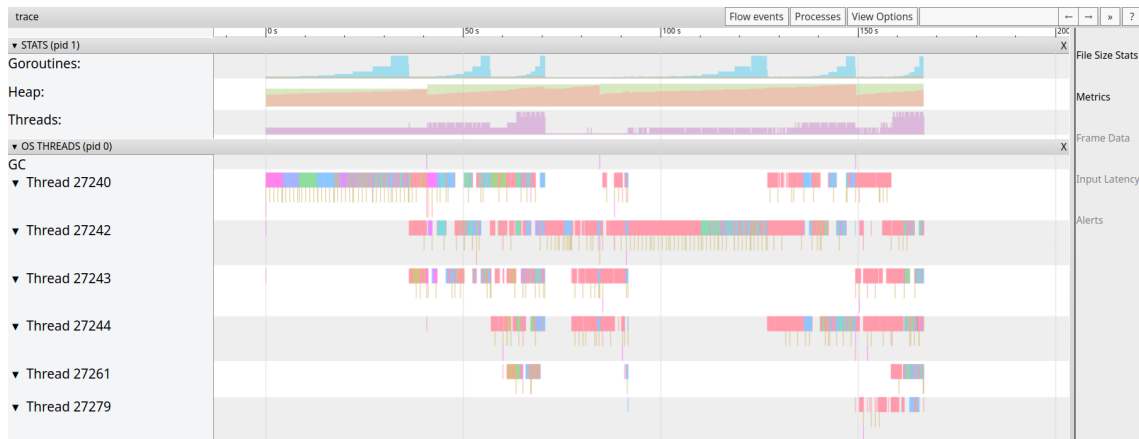
شکل ۱۶: بررسی خروجی trace با توجه به proc

افت ناگهانی، نشان‌دهنده فعالیت (GC) Garbage Collector است که حافظه‌های اضافی را آزاد می‌کند.

خطوط رنگی تعداد تردهای سیستم‌عامل را نشان می‌دهد. می‌بینیم که Proc 0 تقریباً همیشه مشغول است (نوارهای رنگی ممتد)، اما پردازنده‌های دیگر (Proc 1-3) دارای فضاهاى خالی سفید هستند (هر رنگ نشان‌دهنده یک گوروتین خاص است). این نشان‌دهنده این است که بار کاری شما به طور ۱۰۰٪ متوازن توزیع نشده است یا هزینه‌ی هماهنگی بین پردازنده‌ها باعث شده برخی از آن‌ها در زمان‌هایی بیکار بمانند.

Thread by Trace View ۲-۷-۲

در اینجا برخلاف نمای پردازنده که فقط ۴ ردیف داشت، در اینجا تعداد ردیف‌ها بیشتر است (مانند Thread ۱۷۲۷۰ تا Thread ۱۷۲۹۲). این نشان‌دهنده مدل $M : N$ در گو است که در آن چندین گوروتین روی تعداد متغیری از تردهای سیستم‌عامل نگاشت می‌شوند. بخش‌های خالی وسیع در Thread ۱۷۲۷۱ نشان می‌دهد که سیستم‌عامل تردهایی را ساخته اما زمان‌بند گو در آن لحظات نیازی به استفاده از آن‌ها نداشته است. تراکم بالای نوارهای رنگی در Thread ۱۷۲۷۰ نشان‌دهنده ترد اصلی است که بیشترین بار محاسباتی را بر عهده دارد. سایر تردها احتمالاً برای مدیریت فراخوان‌های سیستمی (Syscalls)



شکل ۱۷: بررسی خروجی trace با توجه به thread

یا کارهای مسدودکننده ایجاد شده‌اند.

۳-۷-۲ Goroutine Analysis

در بخش Goroutine Analysis می‌توانیم اطلاعاتی در مورد هر یک از goroutine های موجود به دست آوریم.

حال اگر روی main.worker کلیک کنیم، می‌توان در جدولی مشاهده کرد که هر یک از این goroutine ها زمانشان را در بازه زمانی trace کجا گذرانده‌اند. در عکس ۱۹، بخشی از این جدول را مشاهده می‌کنید.

Block time (chan receive) نشان‌دهنده بلاک شدن گروتین هنگام انتظار برای دریافت داده از کانال jobs است. برای مثال، گروتین ۱۵۵۷ به مدت $2/112 \mu s$ در این وضعیت بوده است.

Block time (sleep) این ستون نشان‌دهنده بلاک شدن به دلیل فراخوان‌های time.Sleep در بارهای کاری IO یا Mixed است. به عنوان مثال، گروتین‌های بالایی حدود ۸/۱ ثانیه را در این وضعیت سپری کرده‌اند.

Block time (sync) مربوط به انتظار برای قفل‌ها (Mutex) است که در اینجا برای همه موارد $0s$ است.

Goroutines

Below is a table of all goroutines in the trace grouped by start location and sorted by the total execution time of the group.

Click a start location to view more details about that group.

Start location	Count	Total execution time
main.worker	2295	3m37.418617791s
runtime.(*traceAdvancerState).start.func1	1	100.084095ms
main.main	1	26.057601ms
runtime.traceStartReadCPU.func1	1	8.275328ms
runtime.gcBgMarkWorker	4	4.618048ms
runtime/trace.Start.func1	1	3.871114ms
runtime.bgsweep	1	992.128μs
runtime.bgscavenge	1	16.128μs
(Inactive, no stack trace sampled)	2	0s

شکل ۱۸: Goroutine Analysis

Goroutine	Total		Execution time	Block time (chan receive)	Block time (sleep)	Block time (sync)	Block time (syscall)	Sched wait time	Syscall execution time	Unknown time
1557	6.183117696s		4.359854215s	2.112μs	1.783486841s	0s	0s	39.774528ms	0s	0s
1811	6.128536128s		4.25951328s	0s	1.834474176s	0s	0s	34.548672ms	0s	0s
1316	6.006591232s		4.174876416s	0s	1.828772672s	0s	0s	2.942144ms	0s	0s
59	5.12108512s		176.363392ms	0s	0s	0s	0s	4.944721728s	0s	0s
58	5.119276352s		176.706688ms	0s	0s	0s	0s	4.942569664s	0s	0s
86	5.116094528s		186.122816ms	0s	0s	0s	0s	4.929971712s	0s	0s
85	5.114219072s		162.72192ms	0s	0s	0s	0s	4.951497152s	0s	0s
84	5.114048704s		165.767104ms	0s	0s	0s	0s	4.9482816s	0s	0s
55	5.112896576s		175.932608ms	0s	0s	0s	0s	4.936963968s	0s	0s
83	5.111345536s		153.337728ms	0s	0s	0s	0s	4.958007808s	0s	0s
82	5.108309376s		156.93856ms	0s	0s	0s	0s	4.951370816s	0s	0s
81	5.105774592s		152.866048ms	0s	0s	0s	0s	4.952908544s	0s	0s
56	5.10374976s		174.280512ms	0s	0s	0s	0s	4.929469248s	0s	0s
80	5.103705664s		157.748224ms	0s	0s	0s	0s	4.94595744s	0s	0s
79	5.10316192s		157.482048ms	0s	0s	0s	0s	4.945679872s	0s	0s
78	5.100395264s		149.09312ms	0s	0s	0s	0s	4.951302144s	0s	0s
77	5.09941568s		153.477248ms	0s	0s	0s	0s	4.945938432s	0s	0s
76	5.097918016s		157.091008ms	0s	0s	0s	0s	4.940827008s	0s	0s
75	5.095594176s		155.079424ms	0s	0s	0s	0s	4.940514752s	0s	0s

شکل ۱۹: بررسی زمان اجرای هر یک از goroutines

۸-۲ جمع‌بندی

در این پروژه اثرات concurrency و scheduling در زبان Go بررسی شد. نتایج نشان دادند که:

- افزایش تعداد goroutine همیشه باعث بهبود عملکرد نمی‌شود.
 - مقدار GOMAXPROCS تاثیر زیادی بر workload های CPU-bound دارد.
 - Scheduler زبان Go در مدیریت workload های blocking عملکرد مناسبی دارد.
 - پس از نقطه مشخصی، scheduler overhead و contention باعث کاهش کارایی می‌شوند.
- همچنین ابزار runtime/trace دید مناسبی از رفتار داخلی، Scheduler goroutine ها و Collector Garbage فراهم کرد.

۳ ساخت سرویس، کانتینری سازی و اجرا در ماشین مجازی

۱-۳ مقدمه

در سیستم‌های توزیع‌شده، سرویس‌ها معمولاً از طریق پروتکل HTTP با یکدیگر ارتباط برقرار می‌کنند. یکی از رایج‌ترین معماری‌ها برای طراحی این سرویس‌ها، معماری REST است. در این بخش، یک Service Web ساده در زبان Go طراحی شد که عملیات محاسباتی را از طریق API ارائه می‌دهد. همچنین برنامه با استفاده از Docker کانتینر سازی شد تا اجرا و استقرار آن روی سیستم‌های مختلف ساده‌تر شود.

۲-۳ معماری سیستم

سیستم شامل یک Server HTTP است که روی پورت ۸۰۸۰ اجرا می‌شود. دو endpoint اصلی تعریف شده‌اند:

• /health

• /compute

۳-۳ Endpoint مربوط به Check Health

این endpoint برای بررسی وضعیت فعال بودن سرویس استفاده می‌شود. در صورت فعال بودن سرور، عبارت OK بازگردانده می‌شود. نمونه درخواست:

```
curl http://localhost:8080/health
```

1

پاسخ:

```
OK
```

1

۴-۳ Endpoint مربوط به Compute

این endpoint عملیات محاسباتی را انجام می‌دهد.

پارامترهای ورودی:

• op

• a

• b

عملیات‌های پشتیبانی‌شده:

• ADD

• SUB

• MUL

• DIV

نمونه درخواست:

```
curl "http://localhost:8080/compute?op=ADD&a=5&b=7" 1
```

پاسخ:

```
{"status":"ok","operation":"ADD","a":5,"b":7,"result":12} 1
```

۵-۳ ساختار Request و Response

در این پروژه، اطلاعات درخواست از طریق Parameter Query دریافت می‌شود.

ساختار پاسخ JSON به صورت زیر است:

{	1
"status": "ok",	2
"operation": "ADD",	3
"a": 5,	4
"b": 7,	5
"result": 12	6
}	7

در صورت بروز خطا:

{	1
"status": "err",	2
"error": "division_by_zero"	3
}	4

۳-۶ اعتبارسنجی ورودی

قبل از انجام عملیات، ورودی‌ها بررسی می‌شوند.
موارد زیر مدیریت شده‌اند:

- نبود پارامترها
- نامعتبر بودن اعداد
- عملیات ناشناخته
- تقسیم بر صفر

نمونه:

```
curl "http://localhost:8080/compute?op=DIV&a=5&b=0"
```

1

پاسخ:

```
{"status": "err", "error": "division_by_zero"}
```

1

۷-۳ Dockerization

برای قابل حمل بودن برنامه، از Docker استفاده شد.

Dockerfile شامل مراحل زیر است:

۱. انتخاب image پایه Go

۲. کپی فایل‌های پروژه

۳. build برنامه

۴. expose کردن پورت

۵. اجرای سرور

۸-۳ Build کردن Image Docker

دستور Build:

```
sudo docker build -t hw1-part3 .
```

1

خروجی مورد انتظار:

```
Successfully tagged hw1-part3:latest
```

1

۹-۳ اجرای Container

اجرای کانتینر:

```
sudo docker run -p 8080:8080 hw1-part3
```

1

خروجی:

```
[server] Starting server on :8080
```

1

۱۰-۳ نمونه تست‌های انجام‌شده

تست جمع:

```
curl "http://localhost:8080/compute?op=ADD&a=5&b=7"
```

1

پاسخ:

```
{"status":"ok","operation":"ADD","a":5,"b":7,"result":12}
```

1

تست تفریق:

```
curl "http://localhost:8080/compute?op=SUB&a=10&b=3"
```

1

پاسخ:

```
{"status":"ok","operation":"SUB","a":10,"b":3,"result":7}
```

1

تست ضرب:

```
curl "http://localhost:8080/compute?op=MUL&a=4&b=6"
```

1

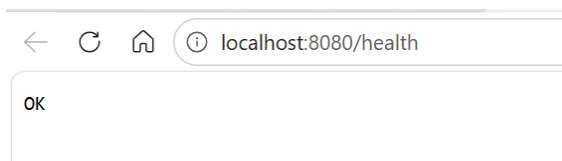
پاسخ:

```
{"status":"ok","operation":"MUL","a":4,"b":6,"result":24}
```

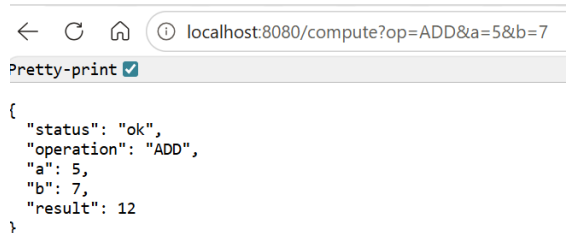
1

تست تقسیم:

۳ ساخت سرویس، کانتینری سازی و اجرا در ماشین مجازی



شکل ۲۰: خروجی health از مرورگر



شکل ۲۱: خروجی op از مرورگر

```
curl "http://localhost:8080/compute?op=DIV&a=8&b=2"
```

1

پاسخ:

```
{"status": "ok", "operation": "DIV", "a": 8, "b": 2, "result": 4}
```

1

۳-۱۱ تست خطاها

پارامتر ناقص:

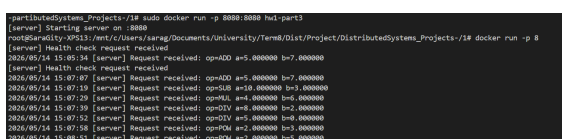
```
curl "http://localhost:8080/compute?op=ADD&a=5"
```

1

پاسخ:

```
missing parameters
```

1



شکل ۲۲: خروجی سرور در طول اجرا

۳ ساخت سرویس، کانتینری سازی و اجرا در ماشین مجازی

```
PS C:\Users\larsg\Documents\University\Term01\DistProject\DistriutedSystem_Projects> curl http://localhost:8888/health
{"status":"ok","operation":"ADD","m":5,"r":{"result":12}}
PS C:\Users\larsg\Documents\University\Term01\DistProject\DistriutedSystem_Projects> curl "http://localhost:8888/compute?theta=delta=1&m=7&r={\"operation\":\"SUB\", \"m\":5,\"r\":{\"result\":12}}\""
{"status":"ok","operation":"SUB","m":5,"r":{"result":12}}
PS C:\Users\larsg\Documents\University\Term01\DistProject\DistriutedSystem_Projects> curl "http://localhost:8888/compute?theta=delta=1&m=7&r={\"operation\":\"MUL\", \"m\":5,\"r\":{\"result\":12}}\""
{"status":"ok","operation":"MUL","m":5,"r":{"result":12}}
PS C:\Users\larsg\Documents\University\Term01\DistProject\DistriutedSystem_Projects> curl "http://localhost:8888/compute?theta=delta=1&m=7&r={\"operation\":\"DIV\", \"m\":5,\"r\":{\"result\":12}}\""
{"status":"ok","operation":"DIV","m":5,"r":{"result":12}}
PS C:\Users\larsg\Documents\University\Term01\DistProject\DistriutedSystem_Projects> curl "http://localhost:8888/compute?theta=delta=1&m=7&r={\"operation\":\"EXP\", \"m\":5,\"r\":{\"result\":12}}\""
{"status":"ok","operation":"EXP","m":5,"r":{"result":12}}
PS C:\Users\larsg\Documents\University\Term01\DistProject\DistriutedSystem_Projects> curl "http://localhost:8888/compute?theta=delta=1&m=7&r={\"operation\":\"LOG\", \"m\":5,\"r\":{\"result\":12}}\""
{"status":"ok","operation":"LOG","m":5,"r":{"result":12}}
```

شکل ۲۳: خروجی client در ترمینال

عدد نامعتبر:

```
curl "http://localhost:8080/compute?op=ADD&a=abc&b=5" 1
```

پاسخ:

```
invalid a
```

عملیات ناشناخته:

```
curl "http://localhost:8080/compute?op=POW&a=2&b=3" 1
```

پاسخ:

```

{"status": "err", "error": "unknown_operation"}

```

۱۲-۳ مزایای استفاده از Docker

استفاده از Docker مزایای مهمی دارد:

- قابلیت اجرا روی هر سیستم
- حذف وابستگی به محیط
- استقرار ساده
- مدیریت راحت نسخه‌ها
- ایزوله‌سازی برنامه

۱۳-۳ جمع بندی

در این بخش، یک Server HTTP کامل در زبان Go طراحی و پیاده سازی شد.
ویژگی های اصلی پروژه:

- طراحی Server HTTP

- استفاده از JSON

- مدیریت خطا

- اعتبارسنجی ورودی

- Dockerization

این پروژه مفاهیم پایه طراحی Web، Service، پیاده سازی API، و استقرار سرویس ها با Docker را به خوبی نشان می دهد.